Faculty of Computing

SE4031 - GD - Assignment 01

Unity

Year 4, Semester 2

Game Documentation

IT22597656 – Ranasinghe S.P

Contents

1. Game Concept Overview 3
2. Implementation of Core Requirements 3
   2.1 Health System 3
   2.2 Score System 4
   2.3 Health Packs 4
   2.4 Projectile Attack 5
   2.5 Enemy Mechanics 5
   2.6 Speed Increment Feature 6
3. Creative Feature 6
   3.1 Start Countdown System 6
   3.2 Dynamic Health Bar Gradient 6
   3.3 Damage Feedback System 7
   3.4 Visual Enhancements 7
4. Controls Guide 8
5. Game Flow 8
6. Code Structure & Stability 9
7. Screenshots 9
8. Assets & Tools Used 15
9. Conclusion 16
10. Appendix 16

1. Game Concept Overview

Tunnel Blaster is a 2D endless side-scrolling action game developed using Unity. The player continuously moves forward through dynamically spawned tunnel obstacles. The objective is to survive as long as possible by:

- Avoiding obstacle collisions
- Passing through tunnel gaps to score points
- Shooting enemies using projectiles
- Collecting health packs to restore health
- Managing increasing speed over time

The game progressively increases in difficulty as the player survives longer.

2. Implementation of Core Requirements

2.1 Health System

The player health system includes:

- Maximum health value of 100
- Health displayed using a UI health bar
- Health decreases:
    - When colliding with obstacles (10 damage)
    - When colliding with enemies (15 damage)
- Player passes through obstacles instead of stopping
- Game ends when health reaches 0
- Health resets to maximum on restart
- Health bar color dynamically transitions from green to red using a gradient based on current health percentage

2.2 Score System

The scoring system includes:

- Score displayed in real-time on UI
- Score increases:
    - When player passes through tunnel gap
    - When enemy is destroyed by projectile
- Final score displayed on Game Over screen
- Animated score count-up effect on Game Over
- Score resets to 0 on restart

2.3 Health Packs

Health packs:

- Spawn randomly near tunnel gaps
- Visually distinguishable from enemies
- Increase player health up to maximum
- Destroyed upon collision with player
- Automatically destroyed after passing camera
- Spawn frequency balanced to prevent infinite survival

2.4 Projectile Attack

Projectile system:

- Left mouse click fires projectile
- Projectiles spawn in front of player
- Velocity based on player speed + base speed
- Destroyed upon collision
- Destroyed after a fixed lifetime
- Shooting disabled during pause, start screen, and game over
- Shooting blocked when clicking UI buttons

Includes sound effects for firing.

2.5 Enemy Mechanics

Enemies:

- Spawn randomly near tunnel gaps
- Visually distinct from health packs
- Destroyed by projectile collision
- Increase score when destroyed by projectile
- Deal damage to player on collision
- Automatically destroyed after passing player
- Play explosion particle effect and sound on death

2.6 Speed Increment Feature

- Player speed increases gradually over time
- Speed capped to prevent excessive difficulty
- Speed scaling is smooth and noticeable
- Makes gameplay progressively challenging

3. Creative Feature

The game includes multiple creative enhancements.

3.1 Start Countdown System

After pressing Start:

- "3…2…1…GO!" animated countdown appears
- Gameplay begins only after countdown completes

3.2 Dynamic Health Bar Gradient

Health bar color smoothly transitions:

- Green → Yellow → Red
  Based on remaining health percentage.

3.3 Damage Feedback System

- Red screen flash when player takes damage
- Fade-in Game Over screen
- Animated final score count-up
- Sound effects for:
    - Shooting
    - Enemy explosion
    - Game over

These enhancements improve immersion and polish.

3.4 Visual Enhancements

Visual improvements include:

- Imported Unity Asset Store sprites
- Distinct themed visuals for:
    - Player
    - Enemies
    - Health packs
- Explosion particle effects
- Trail effect on projectiles
- Fade-in UI animations
- Organized sorting layers
- Polished UI layout

The game avoids default placeholder visuals and uses enhanced assets.

4. Controls Guide

Action	Input
Move Up	W / Up Arrow
Move Down	S / Down Arrow
Shoot	Left Mouse Click
Pause	ESC or Pause Button
Resume	Resume Button
Restart	R or Restart Button
Start Game	Start Button

5. Game Flow

1. Start screen appears
2. Player presses Start
3. Countdown begins (3...2...1...GO!)
4. Gameplay starts
5. Player survives, shoots enemies, collects health
6. Speed increases gradually
7. When health reaches 0:
    - Game Over screen fades in
    - Final score animates
    - Restart option available

6. Code Structure & Stability

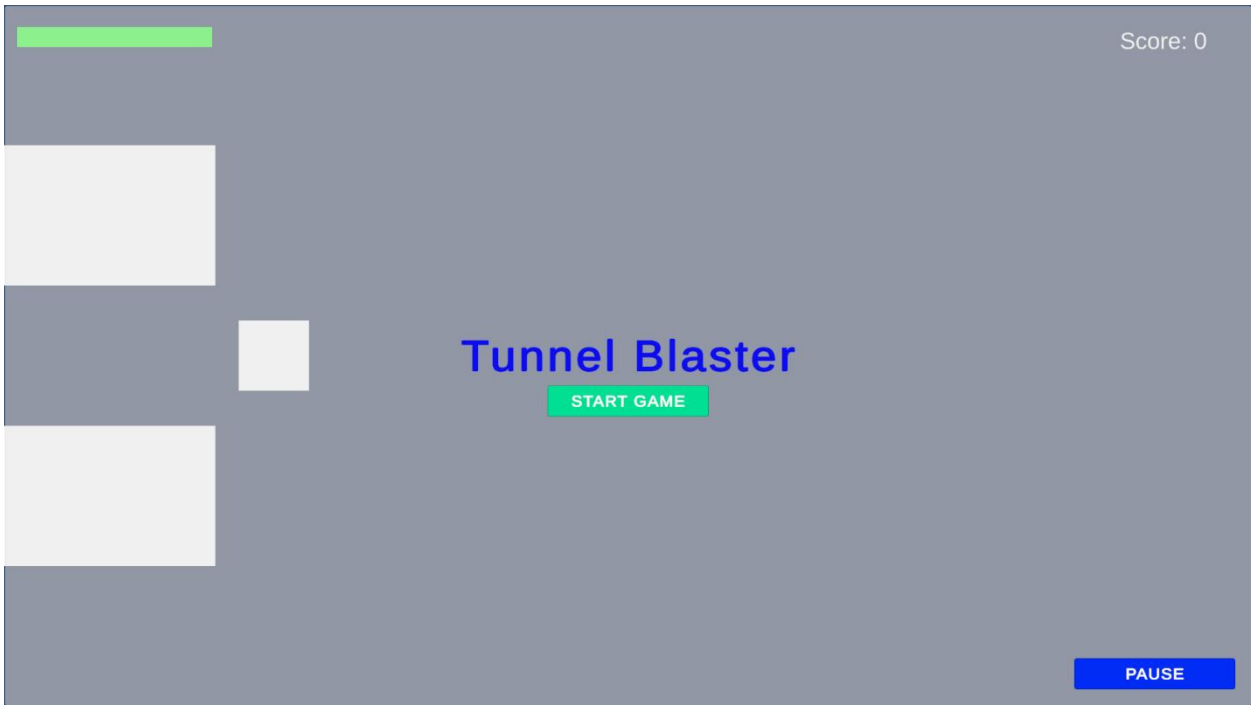
The project follows a modular structure:

- GameManager.cs – controls game state
- PlayerMovement.cs – handles movement and speed
- PlayerShoot.cs – handles projectile logic
- PlayerHealth.cs – manages health and UI
- Enemy.cs – enemy interactions
- HealthPack.cs – health restoration logic
- Spawner.cs – procedural object spawning
- ScoreManager.cs – score management

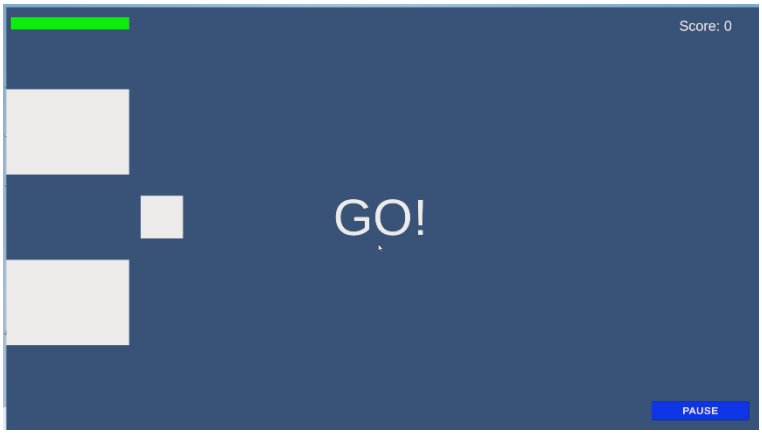
Code is organized and responsibilities are separated logically.

7. Screenshots

1. Start Screen UI

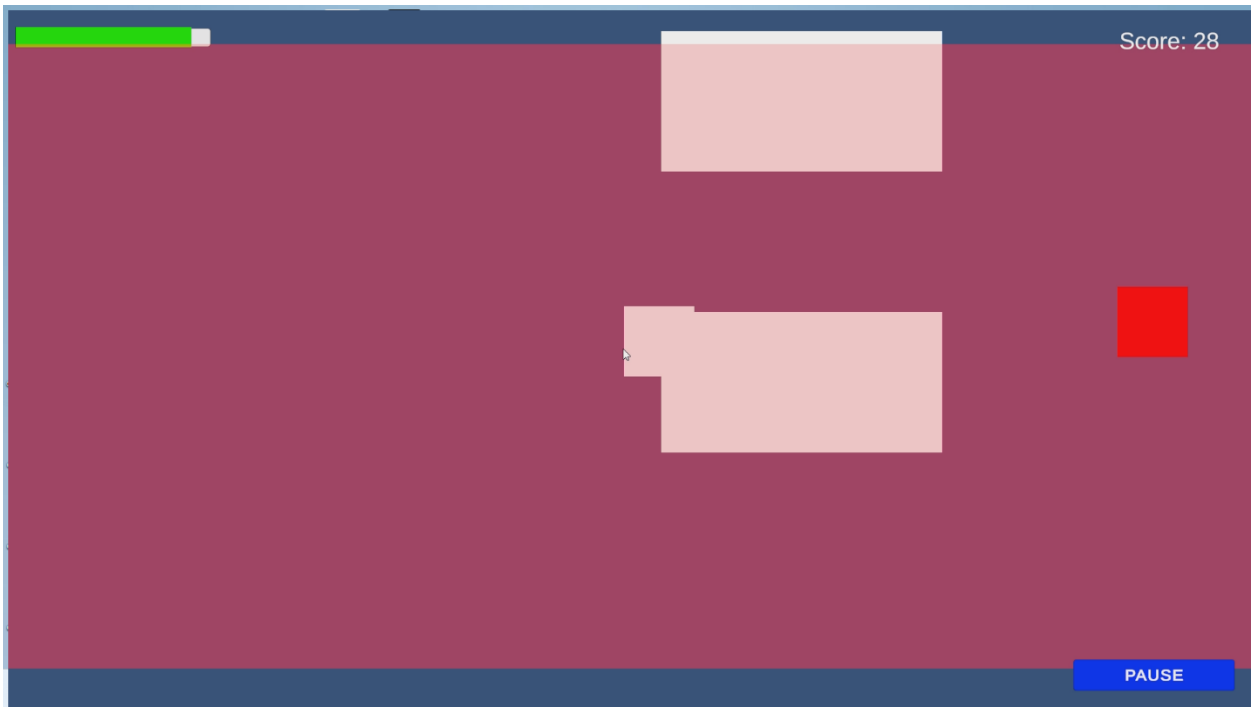


2. Countdown system



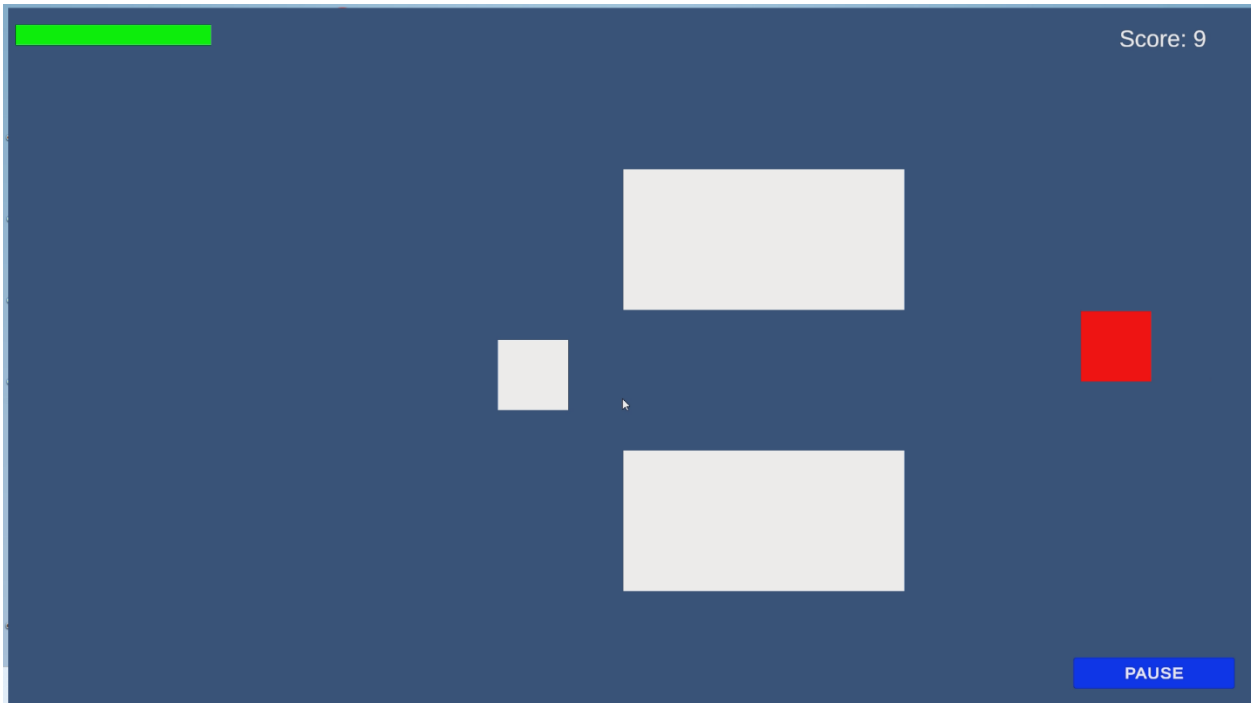
3. Gameplay view

4. Impact UI

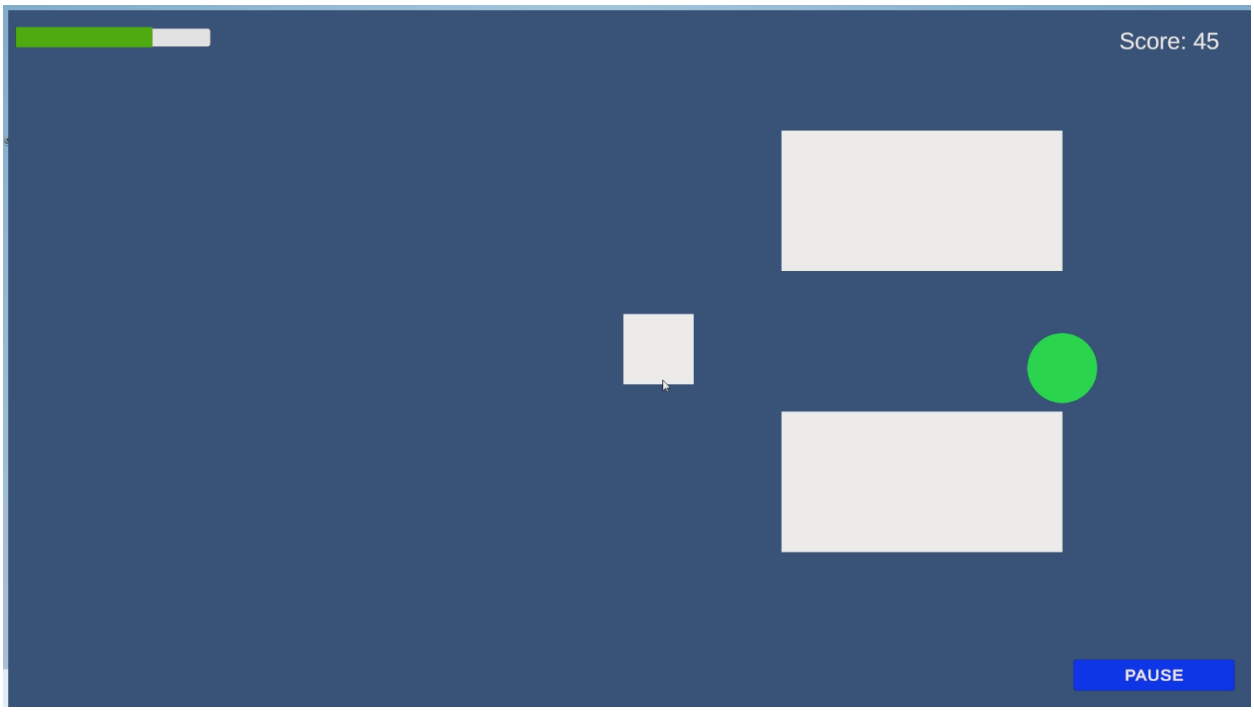


5. Projectile (Yellow)

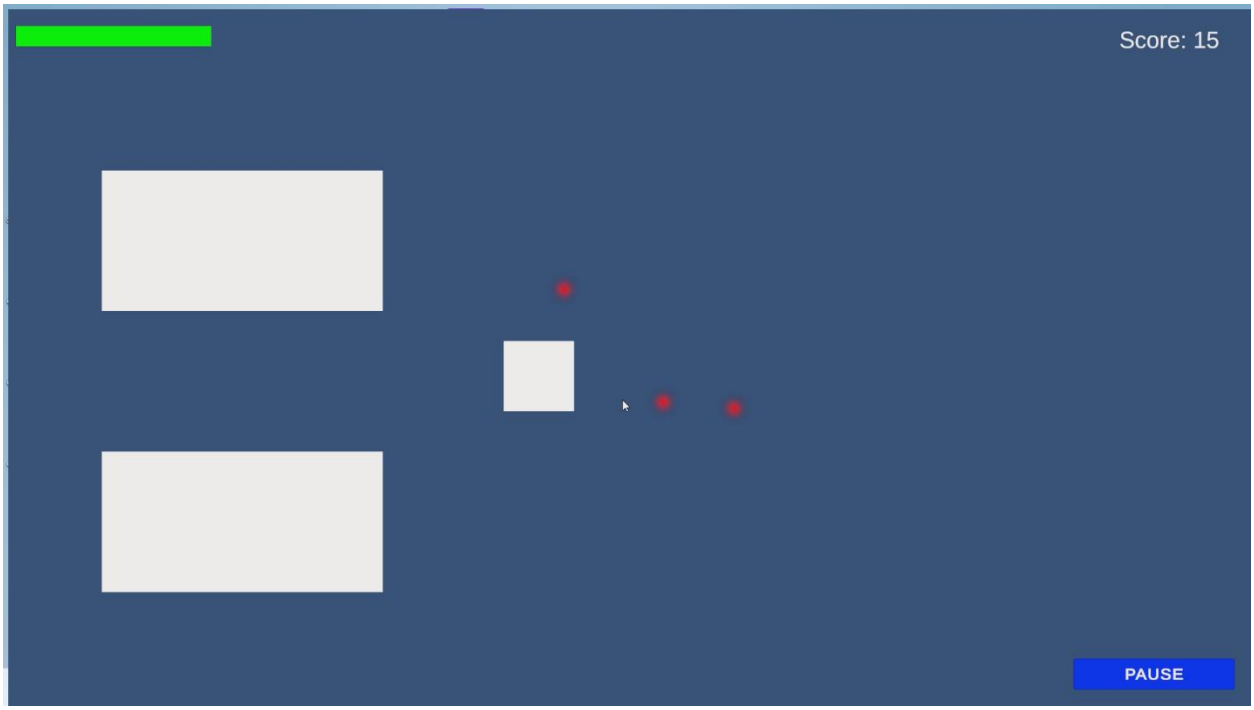
6. Enemy (Red)



7. Health Pack (Green)

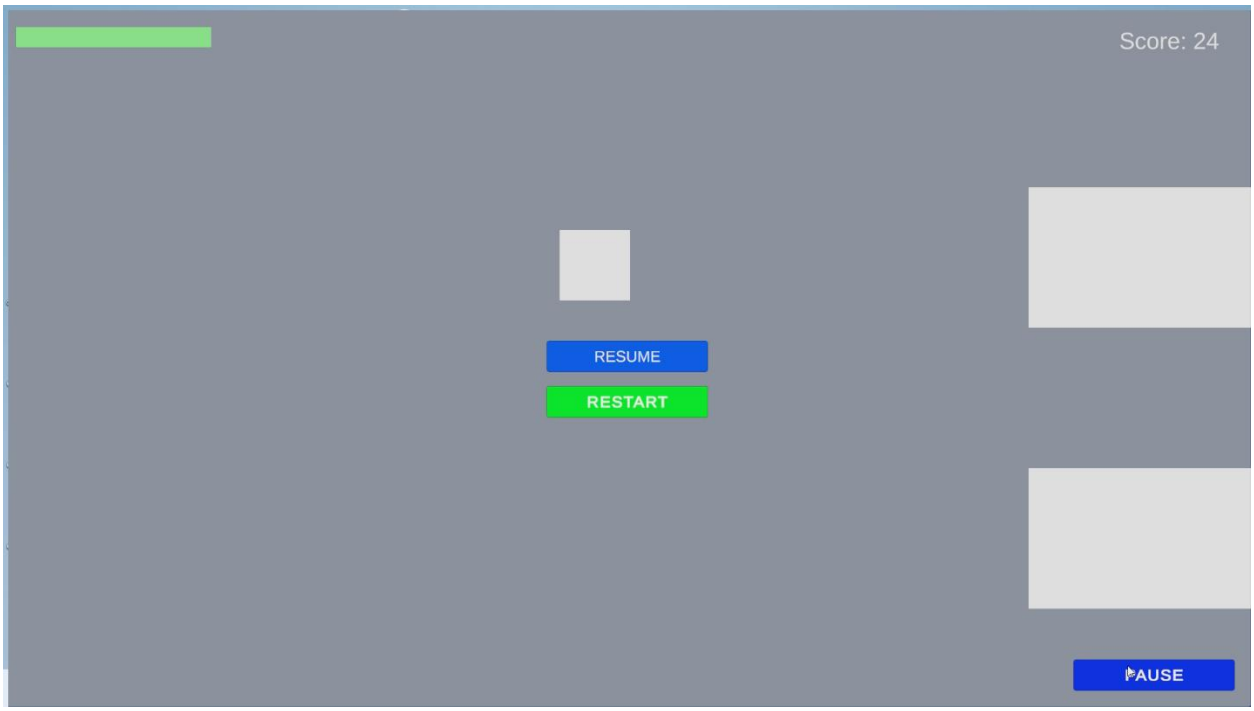


8. Enemy explosion effect



9. Game Over screen with animated score

10. Pause & Restart UI



8. Assets & Tools Used

- Unity 2D Engine
- Unity Asset Store sprites
- TextMeshPro
- Unity Particle System
- Custom audio clips
- C# scripting

9. Conclusion

Tunnel Blaster is a fully developed 2D interactive game created using Unity that demonstrates the practical application of core game development principles. The project incorporates essential gameplay systems such as player health management, scoring mechanics, enemy interaction, projectile functionality, and progressive difficulty scaling.

Additional features including a start countdown, pause functionality, animated score display, and visual feedback effects enhance the overall user experience and presentation quality. The game is stable, well-structured, and implemented using modular C# scripts, reflecting a clear understanding of Unity's 2D development environment.

Overall, the project successfully delivers a functional and polished gameplay experience while demonstrating technical competence in game design and implementation.

10. Appendix

Demo Video Link:

https://drive.google.com/file/d/1-6Qil9eoJnMjxPtP-75x4P2SWzONenvc/view?usp=sharing